

SRW3 Phase 0-D

KEVM Binding — Research Handoff Report

Executable Semantics + KEVM Binding · Reconciliation & Audit Edition —
Theorem-Closure Pass

Headline results

- Claims 1, 2, 3, 4a, 4b: **PROVED** over the abstract SRW3 semantics — no vacuity warnings; negative control correctly fails. Claim 4 is stated only as $4a \wedge 4b$ (the induction step).
- **FULL $\forall n$ SAFETY THEOREM NOW MECHANIZED (§R.1)**: $\text{InitialSafe}(P, \sigma_0) \wedge \text{SecurityClosed}(P) \rightarrow \forall n. \text{Safe}(P, \sigma_n)$ — proved by the ghost-state (invariant-as-data) encoding GH-T2/GH-M reaching #Top via [circularity]; GhostMatches base, accept/reject preservation, certificate-to-real-state soundness and gate agreement all machine-checked; the encoding is a byte-faithful copy of the baseline semantics (507/507 lines, mechanical certificate) plus 13 marked additive lines; the non-vacuity negative control correctly fails.
- **Reconciliation (Priority 1)**: the original gateOk/rootOk placeholders are realized, not replaced — 9 ground claims check the signature-faithful instantiation against the implemented gate; agreements on all covered states, the single divergence class pinned to the wfCheck boundary (§F).
- **Defect lesson (Priority 3)**: the phantom-reject defect is reconstructed and executed — 1 vs 2 krun --search outcomes, accept-side collapse, vacuous coverage (§G).
- **Boundary + binding audits (Priorities 4-6)**: KEVM sources byte-identical to pristine v1.0.921 (778+157 inherited rules vs 29 introduced); stage-by-stage EVM trace with sensitivity probe; lineage classified mechanized-structure vs cryptographic (§H-§J).
- **Full matrix re-run (Priority 7)**: 15/15 Python, 13/13 demos, Claims #Top, negative control fails, KEVM positive/stale/restore — no previous result changed (§K).
- **Round-2 theorem-boundary audit (§Q)**: 4a/4b premises verified as genuine $\text{Safe}(P, \sigma)$ induction-step premises (PROVED; machine-checked equivalences SF0-SF3); the reconciliation restated under the explicit coverage predicate wfCoverage — signature-faithful realization with mechanically checked ground + state-generalized agreement, NOT a refinement theorem (tiers 1-2 PROVED, tier 3 stated, not mechanized); prior-art boundary revised (Phylax Credible Layer: builder/sequencer pre-inclusion enforcement is NOT new); five-level commitment-boundary taxonomy with theorem-to-level binding; full matrix v2 zero regression.
- **Theorem-closure pass (§R)**: $\forall n$ theorem mechanized (§R.1); tier-3 universal reconciliation STATEMENT FORMALIZED; NOT YET MECHANIZED with three verbatim prover boundaries (§R.2); four-level enforcement taxonomy A-D adopted, theorem bound to its level (§R.3); prior-art conjunction survived the freeze check and is frozen (§R.4); 17-item completion classification (§R.5); full matrix v3 zero regression (§R.6); LLVM UNBLOCKED (§R.7).

Item	Status
Toolchain	K v7.1.337 · KEVM v1.0.921 · z3 4.13.3 · plugin 207ae51 (pinned; restored after container restore)
Abstract layer	kompile clean · 13/13 demos · Claims 1,2,3,4a,4b PROVED
KEVM layer	positive COMMIT · stale REJECT+restore · single-segment REJECT+restore

Item	Status
Audit artifacts	bridge (9 claims #Top) · defect pair · induction attempts + verbatim blockers · safe-form audit (SF0–SF3) · generalized bridge (G8–G11 + X3 + necessity probe) · full matrix v2 · ghost $\forall n$ theorem (12 claims; certificate 507/507) · tier-3 attempts · full matrix v3
Evidence labels	PROVED BY K / DEMONSTRATED BY KEVM / ASSUMED / REQUIRES PROTOCOL-CLIENT SUPPORT / NOT YET MECHANIZED (§L)

A. Environment

All components are pinned and installed without root access. The K version is exactly the one KEVM v1.0.921 pins via its `deps/k_release` file. During the audit period the blockchain-k-plugin K sources (lost in a container restore) were re-fetched from GitHub at the pinned SHA and restored into the `kproj` include path; all audit runs used the restored, pinned sources. The KEVM LLVM backend link step requires `libcrypto.a` — skipped; per the audit directive the LLVM retarget is explicitly deferred (§P).

Component	Version	Method	Notes
K Framework	v7.1.337	GitHub .deb, <code>dpkg -x</code>	ubuntu-jammy build
KEVM	v1.0.921	source tag	no binary assets published
blockchain-k-plugin	207ae51	<code>git clone @ pinned SHA</code>	K sources restored; C lib not built (no keccak paths exercised)
Z3	4.13.3	<code>apt-get download + dpkg -x</code>	kprove / Haskell backend
flex / libfl2	2.6.4	<code>apt-get download + dpkg -x</code>	K scanner generation
LLVM runtime	15.0.6	bookworm pool .deb	kore-expand-macros
libsecp256k1	0.5.0	<code>dpkg -x + soname symlink</code>	kore-exec dependency
Java / Python	21.0.12 / 3.12.14	host	preinstalled

B. Commands

```
source /home/z/my-project/tools/env.sh
# abstract layer
kompile srw3.k --backend haskell --main-module SRW3-GATE --syntax-module SRW3-GATE -o hs-out
kprove srw3.k -d hs-out --spec-module SRW3-CLAIMS # Claims 1,2,3,4a,4b -> #Top
kprove proofs/negative_control.k -d hs-out --spec-module SRW3-NEGCONTROL # fails as designed
# reconciliation bridge (Priority 1)
kompile k/compat/srw3-bridge.k --main-module SRW3-BRIDGE --syntax-module SRW3-BRIDGE \
-I k -o k/compat/bridge-out
kprove k/compat/srw3-bridge.k -d k/compat/bridge-out -I k --spec-module SRW3-BRIDGE-CLAIMS
# defect reconstruction (Priority 3)
krun /tmp/honest.srw3 -d proofs/defects/defects-search-out --search # 2 outcomes (defect)
krun /tmp/honest.srw3 -d k/hs-search-out --search # 1 outcome (corrected)
# induction attempts (Priority 2)
kprove proofs/induction_full.k -d proofs/inductB-out -I k --spec-module SRW3-INDUCT-FULL
kprove proofs/induction_record.k -d proofs/inductC-out -I k --spec-module SRW3-RECORD-CLAIMS-PROVED
--claims record-4a
# KEVM layer
krun k/kevm/demos/evm_negative_stale.srw3evm -d k/kevm/kevm-hs-out \
-cMODE=NORMAL -cSCHEDULE=CANCUN -cUSEGAS=false -cCHAINID=1
```

Every command is wrapped by the reproducible audit scripts in `/home/z/my-project/scripts/` (`audit_bridge.sh`, `audit_defect.sh`, `audit_defect_search.sh`, `audit_induction.sh`, `audit_boundary.sh`, `audit_evm_trace.sh`, `audit_matrix.sh`); all transcripts cited below are verbatim command output under `srw3-kevm/transcripts/audit/`.

C. Existing scaffold result — baseline v1.0 VERIFIED (15/15)

The authoritative baseline (`SRW3_Phase_0D_Implementation_v1.0.zip`, sha256 2db8669f...) was ingested verbatim — never reconstructed or modified. It is preserved twice: a read-only pristine copy (`srw3-baseline/`) and a self-contained git work repo (`srw3-work/`) whose main branch is byte-identical to the zip; all work happens on branch `kevm-changes`, so every change diffs against the authoritative baseline. The baseline suite ran FIRST via its canonical Makefile invocation: Ran 15 tests ... OK (exit 0), mapping 1:1 onto the handoff §C expectations. The K scaffold was compiled UNMODIFIED with canonical flags; K v7.1.337 incompatibilities are recorded verbatim (missing MAP/SET/LIST-SYNTAX prelude modules, claims overlay missing requires, Makefile missing `--main-module`), and a minimal recorded patch (`patches/baseline-k-compat.patch`) is applied only on

kevm-changes. The reconciliation of this scaffold is §F.

D. KEVM binding

File	Purpose
k/srw3.k	Abstract SRW3 semantics (SYNTAX/STATE/INVAR/CLOSURE/GATE/CLAIMS) + the five claims
k/kevm/srw3-kevm.k	KEVM binding: SRW3 commands as EthereumSimulation extensions over module EVM + real assembler; mini-driver via #loadProgram/#initVM/#execute
k/compat/srw3-bridge.k	Reconciliation bridge (§F): faithful gateOk/rootOk + gate extraction + 9 ground claims
proofs/defects/phantom_reject.k	Defect reconstruction (§G) + demonstration claims + corrected-side controls
proofs/induction_full.k / induction_record.k	Induction attempts (§F.4): Map-encoded circularity; record-encoded step claims + attempts
k/run_demos.sh, k/demos/*.srw3	13 abstract demos (self-verifying)
k/kevm/demos/evm_*.srw3evm	4 EVM demos (positive, stale-price negative, single-segment gate, setup-only)
proofs/negative_control.k	Deliberately false claim – prover sanity check

Semantic design (no hidden axioms). The commitment boundary: committed and prospective are distinct cells; begin copies $\sigma \rightarrow \hat{\sigma}$; apart from the state-initialization command init (establishing σ_0), the only rule that updates committed is the gate ACCEPT rule (rule census, boundary_census.txt [3]); REJECT discards $\hat{\sigma}$. Obligations are evaluated equations (evalInv) — there is no gateOk(_) => true anywhere; acceptance = allHold(applicable(registry, declaredFootprint), prospective) $\wedge$ member(actor, authorities). The registry is a hypergraph of edge(InvId, AppSet) (+ bareEdge for CM6). wfCheck enforces $\Gamma^* \subseteq \Gamma$ under policy enforce (deliberately unsound trust policy mechanizes §13). securityClosedUniv is parameterized by the obligation universe so the minimal model is closed without vacuous premises. Lineage $\Lambda = \langle \text{Parent, TransitionId, AuthorizationEvidence, PolicyVersion, Child} \rangle$ is appended only by the commit path. The KEVM layer evaluates IOL/ILD/IOLD over real accounts storage; commit re-snapshots; reject restores storages from the snapshot.

Deviations forced by the toolchain. (1) driver.md/optimizations.md outside kompile scope — needed driver mechanics reproduced minimally; (2) KEVM configuration used unmodified — SRW3 state threaded through <k> as #w3State carriers (nested-cell extension of <kevm> is blocked by cell-map unit regeneration in K v7.1.337); (3) contracts at 0x1001-0x1003 (precompile collision caught during bring-up).

E. Proof results

Every claim below was re-verified in the post-reconciliation matrix run (full_matrix.txt [3]) with no vacuity warnings, and the prover is validated by a negative control that kprove correctly fails (exit 113).

#	Claim	Status	Evidence
1	Local preservation – single-app universe; accepted locally-valid transition commits a state satisfying the declared invariant, lineage appended	PROVED	SRW3-CLAIMS claim 1
2	Gate coverage – violating prospective state cannot produce committed state: committed unchanged, prospective discarded, reject	PROVED	claim 2
3	Lineage append – child commit(N), record lin(parent=CH, tid=N, auth(actor), polver, child=commit(N)), head/next advance	PROVED	claim 3
4a	Admissible one-step transition preserves safety – accept commits Safe(IA $\wedge$ IB $\wedge$ IAB); premise verified as genuine Safe(P, σ_n) (§Q.1)	PROVED	claim 4a; premise-form audit (SF1/SF2)

#	Claim	Status	Evidence
4b	Inadmissible one-step transition cannot commit – committed unchanged ($\sigma_{n+1} = \sigma_n$), reject; Rejected(σ_n, τ) verified as the exact accept-condition negation (§Q.1)	PROVED	claim 4b; premise-form audit (SF3)
–	FULL THEOREM: $\text{Reach_committed}(A) \subseteq \text{Safe}(P) - \forall n. \text{Safe}(P, \sigma_n)$ by induction over committed lineage length	MECHANIZED over the ghost-instrumented semantics – GH-T2/GH-M #Top (§R.1; supersedes the §F.4 hand-proof status)	induction_ghost.k + certificate 507/507 + ghost_theorem.txt
NC	Negative control – false claim " $x_A + x_B > 10$ commits"	correctly FAILS	WarnStuckClaimState

Claim 4 is deliberately not overstated: what is PROVED BY K is exactly $4a \wedge 4b$ — the induction step split by gate outcome. The full theorem requires iterating that step along the committed lineage; §F.4 gives the hand induction, the machine-checked counterparts, and the verbatim blockers.

Assumptions (explicit; none hidden). A1: $\text{trueDeps}(\cdot)$ is a manual read-set derivation from the evalInv equations (auditable). A2: commitments are opaque $\text{commit}(\text{Int})$ terms; no hash modeled (§J). A3: $\text{InputEvidence}/\text{ProofEvidence}$ of Λ are symbolic; authorization evidence = authority-closure membership. A4: closure premises are evaluated against the concrete registry term in each claim — for the negative registry the premise evaluates FALSE, so no vacuous proof over the negative model is possible.

Demo verification matrix (all re-run in `full_matrix.txt` [2]): D1 reject (7,3); D2/CM1 accept (7,7); D2b (7,2) chain; D3/CM7 trust commits $\text{appC}:=11$; D3b co-edge defense-in-depth; D4 enforce blocks; D5 reject-auth; D6 honest chain 2 commits; D7 undeclared read blocked; D7b stale price rejected via IOL; D8/D9 IOLD rejects; D10/CM6 pairwise-only accepts three-way violation. EVM layer: evm_positive COMMIT {100,1}{100,5}{4,100} with lineage $\text{price}=100$; $\text{evm_negative_stale}$ REJECT+restore (all empty, $n=0$, $h=-1$); evm_min2 single-segment REJECT+restore.

F. Reconciliation of the original scaffold (Priority 1)

The original baseline scaffold declares two placeholders with no defining equations — $\text{gateOk}(\text{Map}, \text{Map}, \text{List}, \text{Set}, \text{Set}, \text{Map})$ and $\text{rootOk}(\text{Set}, \text{String}, \text{String})$ — and a tau skeleton guarded by $\text{gateOk}(C, S, \text{TR}, R, H, M)$. The implemented semantics did not delete or rename them: it realizes them.

F.1 The scaffold is a specification, not an implementation

Mechanical evidence (`original_scaffold_probe.txt`): (1) the syntax admits no programs — $\text{Tau} ::= \text{tau}(\text{Appld}, \text{TransitionId})$ exists but $\text{Appld}/\text{TransitionId}$ have no productions; (2) even with an inhabitable tau, no transition executes — a probe supplying the tokens still ends with $\text{tau}(\text{appA}, t1)$ unconsumed because gateOk has no equations, so both tau rules' requires are stuck; (3) the skeleton has no commitment semantics — its rules keep committed and shadow unchanged and define no reject path; (4) rootOk is dead — declared, referenced by no rule; (5) the signature is underspecified for its own purpose — no actor/authority argument, no footprint argument, so neither authorization nor $\text{Affected}(\tau)$ is expressible. **The original predicates were underspecified; the implementation is a realization, checked mechanically (F.3).**

F.2 Component mapping

Original	New semantic predicate/rules	Meaning
C:Map (committed cell)	<code>committed : Map StKey-Int</code>	committed state σ (unchanged role)
S:Map (shadow cell)	<code>prospective : Map StKey-Int</code>	prospective state $\sigma^\wedge$ – begin + evaluation on $\sigma^\wedge$ + commit give it an active role the skeleton never specified
TR:List (event trace)	lineage map (Λ records) + footprint-t/d	event list subsumed by Λ ; footprints ADDED – $\text{Affected}(\tau)$ needs $\text{Foot}(\tau)$, which no gateOk argument supplies
R:Set (roots) + $\text{rootOk}(R, s1, s2)$	authorities set + head anchoring; <code>member(Ac, As)</code> ; lineage parent = head	root-anchored authorization: actor inside the root-derived authority closure (reject-auth), every commitment chains to the root

Original	New semantic predicate/rules	Meaning
H:Set (hypergraph)	registry : Set of edge(InvId, AppSet) (+ bareEdge)	same shape, now with obligation identity and app sets expressed
M:Map (contracts)	fused into the edges: edge(InvId, AppSet)	the original separated contracts from the hypergraph with no link; the realization fuses them
gateOk(C,S,TR,R,H,M)	allHold(applicable(REG, FD), P) – the obligation conjunct of ACCEPT	every obligation applicable to this transition holds on the prospective state
skeleton accept/reject (gate flag)	gate ACCEPT / REJECT / REJECT-auth	accept commits σ^+ + appends Λ ; reject discards σ^+ (restore, at the KEVM layer)

F.3 Signature-faithful instantiations, checked by evaluation

```
rule gateOk(_C:Map, S:Map, _TR:List, _R:Set, H:Set, _M:Map)
=> allHold(universeObls(H), S) // ALL obligations carried by the hypergraph
rule rootOk(R:Set, ActorName:String, _Tid:String) => memberName(ActorName, R)
```

gateOk_faithful is the strongest accept condition expressible within the original six-argument signature: with no footprint argument, the only faithful reading checks every obligation in the hypergraph on the prospective state. The implemented gate checks only the applicable obligations. Agreement was verified claim-by-claim (bridge_reconciliation.txt, kprove exit 0, #Top; the claims are pure ground evaluations — a disagreement would evaluate to false and fail):

Claim	State (demo suite)	Faithful	Implemented	Verdict
G1 (D1)	complete registry, (7,3)→(7,7) violating IAB	false	false	AGREE (both reject)
G2 (D6)	chain registry, honest lending commit	true	true	AGREE
G3 (D7b)	chain, stale price 90	false	false	AGREE
G4 (D8)	settlement at 90 (IOLD)	false	false	AGREE
G5 (D2/CM1)	negative registry, (7,3)→(7,7)	true	true	AGREE – CM1 defect is in the registry, not the gate
G6 (D10/CM6)	pairwise-only, three-way hole	true	true	AGREE – same lesson
G7	complete registry, honest (7,2)	true	true	AGREE
X1 (D3 trust)	hidden write appC:=11, declared foot {appB}	false	true	DISAGREE – notBool(≈) PROVED: divergence pinned to effect incompleteness (CM7)
X2 (D4 enforce)	same state, true effects declared	false	false	AGREE restored – wfCheck blocks the divergence

Reconciliation conclusion (terminology fixed by the Round-2 audit, §Q.2). On every state whose declared footprint covers the true effects, the implemented gate decides exactly as the signature-faithful instantiation of the original placeholders; the only reachable divergence class is effect incompleteness, which wfCheck blocks from reaching the gate. The result is a **signature-faithful realization with mechanically checked ground agreement over the covered states** — not a refinement/simulation theorem: finite ground agreement is tier 1 of the three-tier structure (§Q.2), and the fully quantified agreement statement is stated but NOT YET MECHANIZED. Earlier drafts called this a “refinement”; that wording is superseded.

F.4 Full lineage-length safety theorem — hand proof, machine-checked steps, precise blocker

```
InitialSafe_P( $\sigma_0$ )  $\wedge$  SecurityClosed(P)
-----
 $\forall n. \text{Safe}(P, \sigma_n) \text{ hence } \text{Reach\_committed}(A) \subseteq \text{Safe}(P)$ 
```

Hand induction (meta-theorem). Base: InitialSafe_P — σ_0 initialized to a Safe state (evaluated, not assumed; A4). Step: assume Safe(P, σ_n); any one-step transition either (i) is accepted — the accept requires is exactly allHold(applicable(REG, FD), $\hat{\sigma}$) $\wedge$ authority; under the complete registry applicable = {IB, IAB} for a B-footprint write, so IAB holds on $\hat{\sigma}$ by evaluation and IA holds because the untouched x_A is carried from σ_n by the induction hypothesis — hence Safe(P, σ_{n+1})

(**Claim 4a, PROVED**); or (ii) is inadmissible and cannot commit — REJECT leaves committed unchanged (**Claim 4b, PROVED**); by Claim 2 + rule census there is no third way for committed to change apart from initialization. Conclusion: by induction on the committed lineage length n (each commit appends exactly one record — Claim 3), $\forall n$. Safe(P, σ_n). ■

Attempt	Encoding	Result
[B] full theorem as [circularity], safety observer chkS	Map (fully symbolic committed state)	STUCK – program stays PG $\rightarrow$.K; safeMin(M) unevaluable (recursive lookupInt over constructor-free symbolic Map does not reduce); rule applicability decided by evaluation of requires, not entailment against path constraints
[C-a]/[C-b] 4a/4b-shape step claims	record (reachable state space; key sets are transition-invariant)	PROVED (record-4a, record-4b $\rightarrow$ #Top) – the step is machine-checked in a second, fully symbolic-value encoding
[C-attempt-1] one cycle + chk observer, termination form	record	STUCK – residual #Not(?YB #Equals W $\wedge$...): kore destination handling renames the destination variable and loses the rule-produced value linkage; the same shape WITHOUT the observer closes with #Top (isolation experiment)
[C-attempt-2] full theorem as [circularity]	record	STUCK – same residual class at first circular reuse

Blocker statement (precise — §F.4 status, superseded by §R.1). Three stacked reasons, each captured verbatim: (i) symbolic-map lookups over update terms do not reduce; (ii) reachability-logic side conditions cannot be consumed as rewrite hypotheses; (iii) kore destination matching loses rule-produced value linkages. These blocked the [B]/[C] encodings. §R.1 closes the packaging with the ghost-state (invariant-as-data) encoding, which avoids symbolic Map evaluation (concrete-shape committed map), replaces destination value-linkage with certificate data, and uses explicit existential destinations — the theorem is now MECHANIZED over the ghost-instrumented semantics.

G. The phantom-reject defect — a formal-methods lesson (Priority 3)

Recorded verbatim during bring-up (worklog Task 4): "CRITICAL semantic bug found + fixed: reject rules referenced unbound variable P (cells used $_P$) $\rightarrow$ kore treated P as free existential, making reject fire with a phantom state. Fixed to named P bound to actual prospective. All 13 demos re-validated." The audit treats this as a first-class result.

G.1 The defect, reconstructed and executable

The verbatim defective rule text was not preserved in surviving transcripts; proofs/defects/phantom_reject.k is a faithful reconstruction from the recorded description (disclosed in its header), and it reproduces the recorded behavior exactly.

```
// DEFECTIVE (reconstruction): cell matched anonymously; P bound by NOTHING
rule <k> gate => .K ... </k>
<prospective> _P:Map => .Map </prospective>
... <result> _R2:String => "reject" </result>
requires notBool allHold(applicable(R2, FD2), P)

// CORRECTED (as shipped in SRW3-GATE): P bound to the ACTUAL prospective state
rule <k> gate => .K ... </k>
<prospective> P:Map => .Map </prospective>
... <result> _R:String => "reject" </result>
requires notBool allHold(applicable(R, FD), P)
```

Why the original was unsound. kompile accepts the defective rule with only a warning ("Variable 'P' defined but not used"). At rewriting time P is not bound by the pattern, so the backend quantifies it existentially in the rule condition: the rule is applicable whenever SOME state violating the applicable obligations exists — which is always. Rejection is then satisfiable with a phantom state: the gate can reject on the basis of a state that is not the state being committed, and never evaluates the real prospective state.

G.2 Demonstration (verbatim: phantom_defect.txt, phantom_defect_search.txt)

Demonstration	Corrected gate	Defective gate
krun --search, honest (7,3)→(7,2), complete registry	1 outcome: commit, committed (7,2) – deterministic	2 outcomes: commit AND spurious reject with committed (7,3) – phantom path live on an HONEST transition
Claim 2 statement ("violation → reject")	PROVED – reject decided by evaluating the real σ^*	"PROVED" vacuously – phantom fires regardless of the real state; the claim no longer evidences coverage
Claim 4a (verbatim copy)	PROVED	FAILS (WarnStuckClaimState on the phantom reject path) – accept-side claims 1, 3, 4a collapse
False claim "honest transition → reject"	correctly FAILS (no such path)	stuck – the false statement sits on a live execution path

Why this matters. Under the defect the true coverage claim still "passes" — for the wrong reason — while accept-side claims fail; a suite keeping only reject-side claims would look green while evidencing nothing. The defect was caught by demo self-verification (honest transitions nondeterministically rejecting), not by the prover. **Dependence:** Claims 1–4a/4b and the negative control all import SRW3-GATE (the corrected reject rules); re-running the accept-side suite against the defective definition fails, and re-running the false claim against the corrected definition fails — every current result stands or falls with the corrected rule, and stands. **Discipline adopted:** never match a cell anonymously when a requires reasons about its content; treat unbound-variable warnings as errors in gate rules; keep the positive/negative claim pair plus self-verifying demos as permanent regression guards.

H. Commitment-boundary audit (Priority 4)

H.1 Inherited from KEVM vs introduced by SRW3

Census evidence (boundary_census.txt): every KEVM source file used by the binding (evm.md, asm.md, evm-types.md, word.md, data.md, buf.md, gas.md, schedule.md, serialization.md, network.md, hashed-locations.md, state-utils.md) is byte-identical to the pristine v1.0.921 tarball. The binding introduces 29 rules and 15 syntax declarations (all #w3*/SRW3 driver names) against 778 + 157 inherited rules (evm.md + asm.md). No EVM opcode-semantics rule is touched.

Layer	Content	Status
A. Actual KEVM/EVM execution	SSTORE, SLOAD, CALL (+ args/returndata), RETURN, MSTORE, MLOAD, PUSH, STOP under CANCEL; account/storage cells; assembler	Inherited, unmodified (byte-identical sources)
B. SRW3 prospective-state semantics	begin ($\sigma \rightarrow \sigma^*$); declared/true footprints; wfCheck ($\Gamma^* \leq \Gamma^*$); evalInv/applicable/allHold over the prospective state; at the KEVM layer #w3read/#w3IOL/#w3ILD/#w3IOLD over real accounts storage	Introduced by SRW3 (29 rules), reading through KEVM's own cells without modifying them
C. Modeled commitment/restore semantics	Abstract: gate ACCEPT (only transition-result rule updating committed) / REJECT / REJECT-auth; lineage append; head. KEVM: #w3Gate (commit = re-snapshot + lineage), #w3RestoreAll/#w3RestoreOne (reject = write storages back), #w3Snap0	Introduced by SRW3 – a MODEL, not a mechanism: demonstrates what a commitment-point mechanism must do

H.2 The semantic sequence, rule by rule

```

 $\sigma$  --#w3Snap0--> committed snapshot S (threaded in <k> as #w3State(S,...))
 $\sigma$  --KEVM-->  $\sigma^*$  #w3Run(A,OPS) → #w3SetCtx → #loadProgram → #initVM → #execute:
REAL opcodes mutate REAL <account>/<storage> cells (evm.md rules)
 $\sigma^*$  --SRW3 Gate--> commit( $\sigma^*$ ) #w3Gate accept: #w3IOL  $\wedge$  #w3ILD  $\wedge$  #w3IOLD over REAL storage
→ snapshot := post-storage, lineage w3lin(parent=h, tid=n, price)
 $\sigma^*$  --SRW3 Gate--> reject+restore #w3Gate reject → #w3RestoreAll(S) per account:
<storage> := snapshot (prospective EVM effects discarded)

```

This sequence is a **modeled architecture, not an existing Ethereum protocol capability**: stock Ethereum commits transaction effects atomically at block/state-commitment time with no obligation-evaluation hook between execution and commitment; no rule above exists outside this model.

H.3 Where the gate can actually sit — five-level enforcement taxonomy (revised, §Q.4)

The earlier statement “only an execution client or protocol can provide pre-inclusion enforcement” is **superseded**: builder/sequencer-level enforcement exists in practice (Phylax Credible Layer on Linea — §Q.3) and occupies its own level with its own trust profile.

Level	What it can prevent	Trust / bypass / consensus / independent contracts
A. Contract-level	Violations passing through the gated contract’s own entry points	Trust: the contract’s own (audited) code · Bypassable: YES (direct SSTORE, delegatecall, upgrades) · Consensus: no · Independent contracts: NO – only voluntary adoption
B. Compiler/tool chain	Under-declared effects and missing obligation code for anything compiled by that toolchain	Trust: compiler + bytecode provenance · Bypassable: YES (other toolchains, hand-written EVM) · Consensus: no · Independent contracts: partial (homogeneous ecosystem only)
C. Builder/sequencer pre-inclusion	Any tx violating registered assertions submitted through the enforcing builder/sequencer (Phylax model)	Trust: operator + assertion correctness, NOT consensus · Bypassable: YES (other builders, self-building) · Consensus: no (policy) · Independent contracts: YES for its inclusion set
D. Execution-client	Any transition processed by that client (the modeled #w3Gate/#w3RestoreAll sequence)	Trust: node operators / client supermajority · Bypassable: only by not using it (self-fork risk = liveness cost) · Consensus: no unless de facto adoption · Independent contracts: YES for its processed view
E. Protocol/consensus validity	ANY violating state from ever finalizing, network-wide (block invalid unless every applicable obligation holds on the post-state)	Trust: the protocol’s social/consensus layer · Bypassable: NO within the protocol · Consensus: YES · Independent contracts: YES, universally

Finding (theorem-to-level binding). The compositional safety theorem’s assumptions — single commit path, all applicable obligations evaluated on the prospective state before commitment, restore on reject — are satisfied unconditionally and network-wide **only at level E**; the theorem is consensus-strength iff the commitment gate is enshrined in protocol validity. At level C it holds relative to the enforcing builder/sequencer’s inclusion set; at level D, relative to the client’s processed view. The model does not require modifying EVM opcode semantics; it requires a **commitment-point hook** — and the honest statement (§Q.5) is that the hook exists in the model, not in any deployed protocol.

I. EVM binding uses real EVM semantics — execution-path verification (Priority 5)

The obligations must depend on actual EVM-derived state, not injected abstract values. Evidence: command-granularity prefix traces of the stale-price demo (evm_trace_stale.txt) — each stage runs the full KEVM pipeline for the commands executed so far and extracts real cell contents.

Stage	Commands executed	Real cell contents observed
1	#w3Setup #w3Snap0	all three storage cells = .Map; snapshot carrier s = {4097↦.Map, 4098↦.Map, 4099↦.Map}, n=0, h=-1
2	+ oracle segment	oracle storage {0↦100, 1↦1} – written by REAL SSTORE through evm.md rules
3	+ lending segment	lending {0↦90, 1↦5} – REAL CALL to 4097 (gas 10000, returndata 32 bytes to memory[0..32]), then SSTORE of the stale constant 90, ignoring the return buffer
4	+ liquidator segment	liq {0↦4, 1↦90} – REAL CALL to lending, whose code (PUSH 0; SLOAD; MSTORE; RETURN) returned lending slot0 = 90; liq MLOAD(0) read the return buffer and SSTORED it as priceLineage – the stale price flowed oracle → CALL/RETURN → memory → storage, entirely inside KEVM

Stage	Commands executed	Real cell contents observed
5	#w3Gate0	REJECT + RESTORE: #w3IOL() = (lending slot0 == oracle slot0) = (90 == 100) = false on real storage → #w3RestoreAll writes all three storages back to the snapshot; carrier unchanged (n=0, h=-1); no lineage
Sensitivity probe	oracle SSTOREs 90; lending stores the CALL-returned value (MLOAD of the return buffer) instead of a constant	COMMIT: all three storages agree at 90 ({0→90,1→1} / {0→90,1→5} / {0→4,1→90}); lineage w3lin(parent=-1, tid=0, price=90); carrier advanced (n=1, h=0)

Reading. The differential between stage 5 and the probe is the point: the same gate, evaluating the same obligations over real storage, flips from reject to commit precisely when the EVM-derived values make the obligations true — and the committed lineage record’s price field (90 vs 100) is the value that flowed through real CALL/RETURN/MLOAD/SSTORE execution. The positive and negative examples are therefore traced end-to-end through genuine EVM semantics (oracle state → CALL → returned price → lending state → liquidator state → IOL/ILD/IOLD → reject + restore), with no manually injected abstract values anywhere in the gate’s inputs.

J. Lineage audit: mechanized structure vs cryptographic authentication (Priority 6)

What is mechanized (PROVED BY K — Claim 3, plus the single-commit-path census): lineage is a chain of $\Lambda = \langle \text{ParentCommitment}, \text{TransitionId}, \text{AuthorizationEvidence}, \text{PolicyVersion}, \text{ChildCommitment} \rangle$ records appended only by the gate’s accept rule, parent = current chain head, child = commit(next). This mechanizes lineage STRUCTURE: ordering, chaining, one-record-per-commit, authority-evidence membership at creation time. It intentionally provides no cryptographic content: commitments are opaque commit(Int) terms (A2), authorization evidence is auth(AppId) (A3), and nothing binds record contents to the data they summarize. **Symbolic lineage proves structural ordering properties only; it does not prove provenance.** No claim in this report should be read as implying that the mechanized lineage establishes cryptographic authenticity of parents, data, or evidence.

What Verify(parent, data, proof) requires to be cryptographically meaningful — four concrete additions: (1) **Commitment function** — replace commit(Int) with keccak256 (requires the krypto plugin hook AND a built libkrypto — currently not built, §O); head := keccak256(parent ++ tid ++ inputEvidence ++ policyVersion ++ postStateRoot) makes the lineage a tamper-evident hash chain. (2) **Data binding** — data must be the (hash of the) state the record summarizes: bind each record to a state root of the prospective post-state (Merkleized accounts storage), otherwise the chain orders records but says nothing about which state they certify. (3) **Inclusion proof** — Verify must check that a claimed (account, slot, value) is in the committed state: Merkle-Patricia inclusion proofs against that state root, recomputed and compared with the root hashed into parent. (4)

Authorization evidence — replace auth(AppId) with a signature over (parent, tid, data, policyVersion) verifiable against the actor’s public key (secp256k1 already linked for kore-exec; ecrecover unexercised), plus revocation/validity windows if delegation is admitted (CM3/CM5). With 1–4 in place, lineage becomes an authenticated hash chain anchored in state roots; until then: **mechanized lineage structure (PROVED BY K); cryptographically authenticated lineage (NOT YET MECHANIZED).**

K. Full verification matrix — post-reconciliation re-run (Priority 7)

Every row was re-executed after the reconciliation and audit artifacts were added (full_matrix.txt); no previous result changed. A second full re-run after the Round-2 audit is recorded in full_matrix_v2.txt (§Q.6) — same verdict, zero regression. The audit artifacts are additive — no rule in k/srw3.k or k/kevm/srw3-kevm.k was modified — which is why rows 1–7 are expected unchanged, and are confirmed unchanged.

#	Check	Expected	Observed (this re-run)	Verdict
1	Baseline Python suite (make test, canonical)	15/15	Ran 15 tests ... OK, exit 0	unchanged
2	Abstract demo suite (13 demos)	13/13 as-expected	D1–D10, D2b, D3b, D7b all match \$E	unchanged
3	Claims 1, 2, 3, 4a, 4b (kprove)	all PROVED, no vacuity	exit 0, #Top, zero WarnTrivial/WarnStuck	unchanged
4	Negative control	correctly FAILS	exit 113, WarnStuckClaimState	unchanged
5	KEVM positive bundle	COMMIT + lineage	storages {100,1} {100,5} {4,100}; w3lin(parent=-1, tid=0, price=100); n=1, h=0	unchanged
6	KEVM stale-price bundle	REJECT + restore	all storages restored to .Map; n=0, h=-1; no lineage	unchanged
7	KEVM restore (evm_min2)	REJECT + restore	all storages .Map after gate	unchanged
8	NEW: reconciliation bridge (9 ground claims)	agreements + pinned disagreement	kprove exit 0, #Top; G1–G7 agree; X1 pins CM7 divergence; X2 restoration	PASS
9	NEW: defect demonstrations	corrected vs defective contrast	1 vs 2 krun --search outcomes; 4a collapse under defect; vacuous Claim 2 under defect	PASS
10	NEW: induction attempts	step proved or blocker documented	record-4a/4b PROVED; full-theorem attempts stuck with verbatim residuals	PASS

L. Evidence classification and scoped scientific conclusion (Priority 8)

L.1 Status of every major statement, in the mandated vocabulary

Statement	Classification
Claims 1, 2, 3 (local preservation; gate coverage; lineage append)	PROVED BY K
Claim 4a (admissible one-step transition preserves safety)	PROVED BY K
Claim 4b (inadmissible one-step transition cannot commit)	PROVED BY K
Full theorem $\text{Reach_committed}(A) \leq \text{Safe}(P)$	PROVED BY K – mechanized over the ghost-instrumented semantics (§R.1); supersedes the §F.4 hand-proof status
Structural uniqueness of the commit path (apart from initialization)	PROVED BY K (Claim 2) + rule census
Gate accepts/rejects on evaluated obligations (no assumed predicates)	PROVED BY K (§23 discipline; claims evaluate <code>securityClosedUniv</code>)
Refinement of the original scaffold (faithful <code>gateOk/rootOk</code> agreement)	PROVED BY K on all covered states; divergence class pinned to the <code>wfCheck</code> boundary (§F.3)
Real <code>SSTORE/CALL/RETURN/MSTORE/MLOAD/PUSH/STOP</code> ; obligations over real storage; <code>commit/restore</code> on EVM state	DEMONSTRATED BY KEVM (stage traces, §I; negative + sensitivity controls)
Gate mediates cross-application composition over arbitrary contracts	DEMONSTRATED BY KEVM at model scale (3 contracts); not a protocol result
Commitment/restore as an enforcement mechanism	Modeled – DEMONSTRATED BY KEVM inside the semantics; not an Ethereum capability (§H.2)
Obligation registry completeness for a given universe	ASSUMED per model (<code>SecurityClosed</code> is evaluated on the concrete registry; a premise, not a consequence)
<code>trueDeps</code> read-set derivation (A1); opaque commitments (A2); symbolic evidence (A3)	ASSUMED (explicit, §E)
Commitment-point hook in an execution client / protocol	REQUIRES PROTOCOL/CLIENT SUPPORT (§H.3 – concrete modifications per level)
Cryptographically authenticated lineage – <code>Verify(parent, data, proof)</code>	NOT YET MECHANIZED (§J – keccak/libkrypto, state-root binding, MPT proofs, signatures)

Statement	Classification
Full lineage-length induction in K	NOT YET MECHANIZED (§F.4)
CM2 aliasing; CM3 aggregate limits; CM8 hyperproperties; CM9 liveness	NOT YET MECHANIZED (§N scope boundaries)

L.2 What the evidence supports — and what it does not

Supported finding. SRW3 can be represented as a commitment-gated composition semantics: prospective EVM state is evaluated, at a single commitment point, against explicit cross-application obligations (a hypergraph registry + effect-completeness check + authority closure) before becoming committed state; the evaluation is by equations over the actual prospective state; acceptance appends authenticated-structure lineage and rejection restores the prior state. Within the modeled architecture: **Closed-composition enforcement** ⇒ **requires a commitment-point hook** — characterized at the five enforcement levels contract / compiler-toolchain / builder-sequencer / execution-client / protocol (§H.3, §Q.4); the strongest form of the theorem — validity for arbitrary independently developed contracts, network-wide — matches the trust/validity assumptions of the protocol/consensus-validity level (E); at level C it holds relative to the enforcing inclusion set, at level D relative to the client’s processed view.

Explicitly NOT claimed. Ethereum already supports SRW3 (it does not — §H.2); the modeled gate is a real client/protocol mechanism (it is a semantics); pre-inclusion cross-contract invariant enforcement is new (it is not — deployed systems exist, §Q.3); the scaffold reconciliation is a refinement/simulation theorem (it is a signature-faithful realization with mechanically checked agreement, §Q.2); symbolic lineage establishes cryptographic provenance (it does not — §J); the negative-control-validated prover proves more than its stated fragment. [Status update: the full lineage-length theorem, hand-proved at §L-writing time, IS NOW MECHANIZED over the ghost-instrumented semantics per §R.1 — the claim is scoped exactly to that encoding, whose faithfulness is mechanically certified.] Each ingredient is classical; the phase’s specific contribution is the **conjunction** defined in §Q.5, and the audits above are what bound it honestly.

M. Relation to prior art — REVISED in the Round-2 audit (§Q.3)

This section is superseded by §Q.3, which adds current pre-inclusion/runtime invariant-enforcement systems (in particular the Phylax Credible Layer, deployed on Linea), distinguishes five enforcement levels, and reframes the novelty question as a conjunction. Pre-inclusion cross-contract invariant enforcement itself is NOT claimed as new. The table below is retained for the areas already covered at Phase-0 time.

Prior area	What it covers	What SRW3-phase-0 adds (mechanized here)
Solidity verification (SMTChecker, VerX, Securify, Manticore, Echidna, SmartCheck)	per-contract invariants / temporal specs, pre-deployment	cross-application obligations as first-class hyperedges with a closure predicate, checked at a commitment point over composed prospective state
Proof-Carrying Code / Proof-Carrying Smart Contracts	evidence carried with code, checked at load	evidence is state-lineage-linked (parent/child commitments) and re-checked per transition against the current composed state, not once at load
Move / Cadence resource semantics	linear resources prevent duplication/loss inside a language	obligations over storage-level cross-contract dependencies (no language boundary), incl. three-way (non-pairwise) properties (IOLD; CM6)
Capability systems	authority mediation of operations	mediation of state commitment plus authority evidence in lineage (reject-auth)
Universal Composability	ideal-world composition theorem	executable gate-level analogue with explicit closure premises over a concrete semantics; not a UC theorem
Runtime monitoring / firewalling	post-hoc violation detection or mediating wrapper	the pre-commitment distinction (§H): the violated state is discarded, not reported — mechanically demonstrated
Cross-chain security	bridge-level assumptions	orthogonal; lineage records could anchor cross-chain evidence — future work

N. Countermodel disposition

CM	Statement	Disposition
CM1	local invariants true, interaction invariant false	Handled by closure assumption – negative registry (D2) commits the violation; complete registry rejects (D1); InteractionComplete evaluates FALSE on the negative registry – no vacuous proof claimed
CM2	shared-resource aliasing	Requires extension (Phase-1): state is (app, slot)-addressed; aliasing across names needs an explicit aliasing relation – NOT YET MECHANIZED
CM3	authority aggregation/delegation exceeds global limit	Partially handled: authority-closure membership + reject-auth (D5); aggregate limits need threshold obligations – NOT YET MECHANIZED
CM4	source-valid but wrong/stale lineage input	Handled at the interaction level (D7b/D8 via IOL/IOLD over real values); cryptographic binding is A3 – NOT YET MECHANIZED (§J)
CM5	temporal/revocation ordering	Structurally handled (parent = current head; ordering by lineage chain); revocation with validity windows = extension
CM6	three-way interaction not captured by pairwise contracts	Mechanized – D10 (pairwise-only accepts the bad settlement; bareEdge marks the known-but-uncontracted dependency); complete registry rejects (D8/D9)
CM7	hidden effect / dependency omitted from abstraction	Mechanized – D3 (trust commits violation), D4 (enforce blocks), D7 (undeclared read blocked); AND pinned at the reconciliation level: bridge claim X1 is exactly this divergence (§F.3)
CM8	hyperproperty boundary	Out of scope (§29) – not mechanized
CM9	liveness treated as safety	Out of scope (§29) – not mechanized

O. Limitations and threats to validity

1. **Scale of the mechanized model** — claims are proved over a minimal 2-app (+1 hidden-effect app) universe with concrete registries; the closure premise is evaluated, not assumed, but scaling to arbitrary registries needs quantified premises and a different prover strategy. 2. **The induction is not mechanized** — the full lineage-length theorem is hand-proved with machine-checked steps; the three prover-level blockers are documented verbatim (§F.4). 3. **KEVM scope** — SSTORE/SLOAD/CALL/RETURN/MSTORE/MLOAD/PUSH/STOP under CUNCUN with useGas=false; gas accounting, REVERT-gate interaction, CREATE/SELFDESTRUCT, crypto precompiles unexercised (libcrypto not built; no demo needs keccak). 4. **Commitment-point deployment** — the hook does not exist in stock Ethereum clients; it is the finding (§H), not a defect of the experiments. 5. **Defect reconstruction fidelity** — the phantom-reject demonstration uses a reconstruction from the recorded worklog description (the verbatim original rule text was not preserved); the reconstruction reproduces the recorded behavior exactly and both the file header and the transcript disclose this. 6. **Engine-specific lessons recorded**: single-letter tokens collide with builtin variables; syntax-declaration argument names must be lowercase (uppercase breaks the inner parser — re-confirmed during the induction work); [lemma]/[simp] are not recognized by this K version — trusted simplifications use [simplification]; claims must not appear in the --main-module closure; SET.intersection is not reliably evaluated by the Haskell backend; projection casts {:>Int} are opaque to the simplifier; nested-cell extension of <kevm> regenerates cell-map units. 7. **kore destination-variable handling** — the ?YB residual (§F.4) is a prover-behavior finding, not a logic error; flagged for upstream attention.

P. Next steps and remaining blockers

Immediate (per audit directives — completed in the two audit rounds): gateOk/rootOk reconciliation (§F, §Q.2); Claim 4a/4b separation, induction audit and premise-form verification (§F.4, §Q.1); phantom-reject defect lesson (§G); commitment-boundary audit and five-level taxonomy (§H.3, §Q.4); EVM-binding verification (§I); lineage classification (§J); full matrix re-runs (§K, §Q.6); scoped conclusion and prior-art revision (§L, §M, §Q.3, §Q.5).

Secondary (deferred by directive): (1) LLVM-backend retarget of the KEVM binding + driver.md integration — deferred until BOTH audit rounds are consumed; no result depends on it. (2) Close the induction packaging: ghost-state invariant maintenance, a prover upgrade for destination-variable

handling, or quantified frame reasoning — any route must keep the theorem’s statement intact. (3) Close the generalized-agreement theorem: mechanize $\forall H \forall FD \forall S$ agreement (§Q.2 tier 3) — needs structural induction over symbolic sets, or per-instance expansion. (4) Mechanize CM2 (aliasing) and CM3 aggregate limits; CM8/CM9 need a hyperproperty-level decision first. (5) Cryptographic lineage (§J): build libkrypto $\rightarrow$ keccak commitments; state-root binding; MPT inclusion proofs; signature-based authorization evidence; then state and prove `Verify(parent, data, proof)` soundness in K. (6) Declarative registry format generating `edge(...)` registries (the Γ side), closing the loop with a compiler-layer story. (7) Upstream report: file the [lemma]/[simp] attribute findings and the ?YB destination residual against K v7.1.337 with the verbatim transcripts.

Remaining blockers (concise list — updated by §R): full-theorem ($\forall n$ safety) mechanization — RESOLVED by §R.1 (ghost-state encoding; GH-T2/GH-M #Top; the §F.4 packaging blockers are documented as the kore behaviors that the successful encoding avoids). Generalized agreement theorem ($\forall H \forall FD \forall S$) — STATEMENT FORMALIZED; NOT YET MECHANIZED (§R.2): blocked by structural induction over symbolic sets (three verbatim residuals); state-generalized instances ARE mechanized (tier 2). Cryptographic lineage — blocked by the libkrypto build (resource ceiling) plus the §J extension work. CM2/CM3/CM8/CM9 — blocked by model-extension decisions, not tooling. Protocol-level relevance — requires the commitment-point hook at enforcement level C/D (§R.3); a deployed builder/sequencer-level instantiation exists commercially (§Q.3) but does not make SRW3’s theorem consensus-strength.

Q. Round-2 theorem-boundary audit — verdicts, generalization, prior art, taxonomy, contribution

Mandated by the follow-up audit directive: resolve the theorem-level issues BEFORE any Phase-1 design work and BEFORE any LLVM backend work. Every verdict is backed by a re-runnable artifact and a verbatim transcript; the full regression matrix (§Q.6) confirms nothing previously established changed.

Q.1 Claims 4a/4b as induction steps — VERDICT: PROVED (genuine Safe(P,σ) premises)

Audit question: do 4a/4b use a genuine predecessor-state safety premise `Safe(P, sigma)` (or an equivalent predicate), or merely an `InitialSafe(...)` premise restricted to the designated initial state?

Verdict: PROVED — the premises are genuine induction-step premises

(proofs/safe_form_audit.k, transcript safe_form_audit.txt, kprove exit 0, #Top, 4/4 claims):

#	Evidence	Content
1	No <code>InitialSafe</code> predicate exists in the mechanized claims	The symbol appears in <code>k/srw3.k</code> only in a comment stating the base constraint is evaluated, not assumed; <code>X</code> , <code>Y</code> are universally quantified symbolic Ints — nothing pins them to initial values
SF0	IC vacuous on the two-key shape	never-written slot reads 0 (EVM <code>SLOAD</code> analogue) — the <code>minInvs</code> universe reduces to <code>IA</code> , <code>IB</code> , <code>IAB</code>
SF1	Premise $\equiv \text{Safe}(P, \sigma_n)$	the inlined constraint $X \leq 10 \wedge Y \leq 10 \wedge X+Y \leq 10$ is identically the named predicate <code>safeForm({appA$\mapsto$X, appB$\mapsto$Y})</code> — the same predicate the induction observer <code>chkS</code> uses — for ALL symbolic <code>X</code> , <code>Y</code>
SF2	<code>Accepted(σ_n, τ)</code> is the gate’s own accept condition	4a’s conjuncts $V \leq 10 \wedge X+V \leq 10$ are identically <code>allHold(applicable(completeRegistry(), {appB}), $\sigma^{\wedge}$)</code> — the evaluated applicable-obligation conjunction of the <code>ACCEPT</code> rule
SF3	<code>Rejected(σ_n, τ)</code> is the exact negation	4b’s disjunction $V > 10 \vee X+V > 10$ is identically <code>$\neg$allHold(...)</code> , and 4b’s RHS pins $\sigma_{n+1} = \sigma_n$ — exactly the mandated step shape

The remaining premises (`securityClosedUniv`, `authorization`) are theorem-level system premises, not restrictions to an initial state. Joint-soundness note: 4a and 4b together pin the gate’s accept condition exactly — a weakened `ACCEPT` rule fails 4b; a strengthened one fails 4a. **Scope precision:** the verdict covers the STEP statements over the minimal model; the full theorem $\forall n. \text{Safe}(P, \sigma_n)$ remains hand-proved with machine-checked steps, packaging NOT YET MECHANIZED — the Round-2 audit confirms the blockers are a packaging boundary, NOT a premise defect. No theorem statement was changed; nothing was silently patched.

Q.2 Generalizing the scaffold reconciliation — coverage predicate + three tiers

Audit question: can $\text{wfCheck}(\text{state}) \rightarrow \text{gateOk_faithful}(\text{state}) = \text{implementedGate}(\text{state})$ be proved as a general property? **Answer: not in that form — the exact formulation requires an explicit coverage predicate, and the fully quantified statement is NOT YET MECHANIZED.** $\text{wfCheck}(\Gamma^* \subseteq \Gamma)$ is necessary but NOT sufficient: a state can have all true effects declared while an obligation on an untouched application is already violated in the inherited state. The precise predicate is $\text{wfCoverage}(H, FD, S) := \text{for every edge}(J, Xs) \in H: (Xs \cap FD \neq \emptyset \vee \text{evalInv}(J, S))$ — “every obligation the declared footprint fails to make applicable is already satisfied by the state”. Under it: $\text{gateOk} \wedge \text{rootOk} == \text{gateAccept}$ (sketch: $\text{applicable}(H, FD) \subseteq \text{universeObls}(H)$; allHold over the superset factors into allHold over the subset AND the skipped conjuncts, each made true by wfCoverage ; authorization conjuncts coincide under the name-coincidence premise).

Tier	Statement	Status
1 – finite ground agreement	G1–G7 agreements + X1 divergence + X2 restoration over demo states (<code>k/compat/srw3-bridge.k</code>)	PROVED BY K (checked by evaluation, 9/9)
2 – state-generalized agreement on covered instances	G8–G11 (<code>proofs/generalized_bridge.k</code>): <code>complete/{appB}</code> , <code>chainComplete/{lending}</code> , <code>chainComplete/{liq}</code> , <code>negative/{appB}</code> ; agreement for ALL symbolic state values satisfying the wfCoverage instance (the requires IS the coverage predicate, evaluated)	PROVED BY K (4/4, #Top; checked-by-evaluation with universal quantification)
3 – fully quantified agreement ($\forall H \forall FD \forall S$)	stated with wfCoverage	stated, NOT YET MECHANIZED – recursive functions over symbolic aggregates do not reduce; needs structural induction over symbolic sets, beyond the Haskell backend’s reachability+SMT fragment (same boundary as §F.4 attempt [B])

Necessity of the coverage premise (falsification-validated). Ground witness X3: an unauthenticated-but-consistent oracle state (`oracle auth = 0`, all applicable obligations true) — the implemented gate (footprint `{lending}`) ACCEPTS while the faithful instantiation REJECTS; X3 PROVED pins the disagreement. Symbolic probes N1/N2 (G8/G9 with the coverage premise dropped) correctly FAIL (`kprove exit 1`, `WarnStuckClaimState` — `generalized_bridge_negative.txt`): the premise is load-bearing. (For the complete registry over the naturals, IAB entails the skipped IA; the chain-registry witness X3 is natural-valued, which is why it is the canonical one.)

Terminology (mandated, adopted). The reconciliation result is a “signature-faithful realization with mechanically checked ground agreement over the covered states” — extended by tier 2 to state-generalized agreement on the covered (registry, footprint) pairs. It is NOT called a general refinement/simulation theorem. **Link to the safety theorem:** on the ENFORCE system’s reachable states, wfCoverage holds by the frame argument — a transition writes only keys of apps in $FT \subseteq FD$, so every skipped edge’s keys are untouched and $\text{evalInv}(J, \delta) = \text{evalInv}(J, \sigma_n)$, which the induction hypothesis $\text{Safe}(P, \sigma_n)$ makes true; under policy trust the link breaks and X1 is the divergence.

Q.3 Prior-art boundary — revised (pre-inclusion enforcement is NOT new)

Method: 19 targeted web searches + primary-source reads (docs, whitepaper repo, product pages), 2026-09-27; raw evidence archived in `tool-results/prior-art/` (SUMMARY.md + JSON captures).

Mandated correction: pre-inclusion cross-contract invariant enforcement exists in practice and must not be claimed as SRW3 novelty. Reference system: the **Phylax Credible Layer** — developers write assertions in Solidity (`credible-std`) defining “states your protocol should never reach”, with triggers on function calls / storage / balance changes and no contract modification; a PhEVM simulation takes pre/post snapshots and runs the assertions; violating transactions are “dropped during block building”; deployed on Linea (site-reported “\$2.6M+ in 0x drain attempts stopped”). Enforcement level: builder/sequencer (level C of §Q.4) — policy enforcement by the asserting network, NOT consensus validity, bypassable outside its inclusion set.

Enforcement class	Systems surveyed	Relation to SRW3
Post-hoc monitoring (–)	Forta network (alerts after inclusion); HighGuard (cross-chain runtime monitoring)	detects, does not prevent; SRW3’s gate is pre-commitment (D1 distinction)
Contract-level runtime checks (A)	Solidity require/modifier patterns; Scribble (Certora) – source-annotated, compile-time-woven runtime assertions	per-contract scope; no cross-application closure
Compiler/toolchain (B)	Scribble’s instrumentation; SolCMC (CAV 2024) and Solidifier – static cross-contract model checking pre-deployment	static or woven; no commitment-point semantics, no lineage, no runtime gate
Builder/sequencer pre-inclusion (C)	Phylax Credible Layer (Linea); ERC-4337 bundler/paymaster policies (per-UserOp scope)	SAME enforcement level as SRW3’s modeled gate – but assertion-per-protocol: no interaction-hypergraph closure calculus, no effect-completeness predicate, no lineage ledger, no mechanized compositional theorem
Execution-client (D)	modified-client enforcement (Phylax’s L2 sequencer integration is the deployed analogue)	SRW3’s modeled #w3Gate sequence is exactly this shape; nothing deployed on mainnet L1 does it
Protocol/consensus validity (E)	enshrined-invariant discussions (EIP/debate level only)	no general cross-contract obligation gate enshrined anywhere as of the search date
Adjacent building blocks	ZK coprocessors (Axiom, Brevis, Lagrange); Move VM on-chain bytecode verifier (resource safety at VM entry)	evidence/verification substrates, not commitment-gated composition semantics

Revised research question (replaces any novelty claim on enforcement). Not “is pre-inclusion enforcement new?” — it is not. The question is whether SRW3 contributes something specific in the conjunction of: explicit interaction hypergraph; closure completeness (interaction/contract/effect/lineage/gate); effect-completeness (wfCheck); commitment-gated composition (single commit path, restore-on-reject); lineage; mechanized semantics over KEVM; and a compositional safety theorem. No surveyed system combines these ingredients; the closest single systems are Phylax (enforcement level), Scribble (obligation language), SolCMC (cross-contract semantics, static), and Move (VM-level enforcement, language-internal properties). The conjunction claim is scoped to the survey above and its date.

Q.4 Commitment-boundary taxonomy — revised (summary)

The full five-level taxonomy (A contract / B compiler-toolchain / C builder-sequencer / D execution-client / E protocol-consensus, each with what it can prevent, who must trust it, bypassability, consensus-criticality, and independent-contract coverage) is §H.3. The superseded statement “only an execution client or protocol can provide pre-inclusion enforcement” is retracted there. **Theorem-to-level binding:** the compositional safety theorem’s assumptions are satisfied unconditionally and network-wide only at level E; at level C the theorem holds relative to the enforcing inclusion set; at level D relative to the client’s processed view. The strongest theorem is therefore tied specifically to level E, and SRW3’s demonstrated semantics is the shape of the level-D/E hook, not a deployment claim.

Q.5 Updated scientific contribution statement

Supported (scoped) finding. SRW3 is representable as a commitment-gated composition semantics: prospective EVM state is evaluated at a single commitment point against explicit cross-application obligations — hypergraph registry, effect-completeness check, authority closure — before becoming committed state; acceptance appends structurally authenticated lineage; rejection restores the prior state. Formal result within the model: Closed-composition enforcement $\Rightarrow$ requires a commitment-point hook.

Conjunction contribution (replaces ingredient or enforcement novelty claims). The phase’s contribution is the mechanized combination of: interaction-hypergraph obligations; closure completeness as an evaluated premise; effect-completeness (wfCheck); commitment-gated composition; lineage; and the KEVM-grounded execution layer — together with the compositional safety theorem (step PROVED BY K: $4a \wedge 4b$ with genuine Safe(P,σ) premises, §Q.1; full $\forall n$ theorem hand-proved with machine-checked steps and precise verbatim blockers, §F.4) and the

countermodel discipline CM1–CM9. The reconciliation to the original scaffold is a signature-faithful realization with mechanically checked ground + state-generalized agreement under wfCoverage (§Q.2) — not a refinement theorem. Pre-inclusion enforcement itself is prior art (§Q.3); “Ethereum already supports SRW3” remains explicitly NOT claimed (§H.2); the gate becomes a real mechanism only with a commitment-point hook at enforcement levels C/D/E (§Q.4), which is **REQUIRES CLIENT/PROTOCOL SUPPORT**, not a result of this phase.

Evidence-label discipline (§L.1) is unchanged and now also governs §Q: all new mechanized statements (SF0–SF3, G8–G11, X3) are **PROVED BY K** (checked-by-evaluation class, universal quantification documented); tier-3 agreement, $\forall n$ packaging, cryptographic lineage, CM2/CM3/CM8/CM9 remain **NOT YET MECHANIZED**; deployment claims remain **REQUIRES CLIENT/PROTOCOL SUPPORT**.

Q.6 Regression matrix v2 — nothing changed

#	Check	Result (full_matrix_v2.txt)
1	Baseline Python suite (make test, canonical)	Ran 15 tests ... OK – exit 0
2	Abstract demo suite	13/13 as-expected – exit 0
3	Claims 1, 2, 3, 4a, 4b (kprove)	exit 0, #Top, no vacuity
4	Negative control	exit 113 – correctly fails
5	Reconciliation bridge (9 ground claims)	exit 0, #Top, 9/9 checked-by-evaluation (fresh kompiled)
6	Phantom-reject defect audit	corrected = deterministic commit; defective = spurious 2nd outcome (commit + phantom reject)
7a	Safe-form audit (SF0–SF3)	exit 0, #Top (fresh kompiled)
7b	Generalized bridge (G8–G11 + X3)	exit 0, #Top (fresh kompiled)
7c	Necessity probe (coverage premise dropped)	exit 1 – correctly fails (WarnStuckClaimState)
8	KEVM positive / stale / min2	positive = COMMIT {oracle:{0:100,1:1}, lending:{0:100,1:5}, liq:{0:4,1:100}} + lineage; stale = REJECT + full restore + no lineage + head -1; min2 = REJECT + restore (same cell contents as Round-1)

No previously established result changed.

R. Theorem-closure pass — the $\forall n$ safety theorem **MECHANIZED** over the ghost representation

Mandate: close the theorem boundary before any Phase-1 design or LLVM work, using the prescribed invariant-as-data / ghost-state strategy, without weakening any statement and without introducing unproved axioms. Every claim was re-run for this section (transcripts/audit/ghost_theorem.txt, tier3_universal.txt, full_matrix_v3.txt).

R.1 Primary objective — full lineage-length safety theorem: **PROVED BY K**

Target (unchanged): $\text{InitialSafe}(P, \sigma_0) \wedge \text{SecurityClosed}(P) \rightarrow \forall n. \text{Safe}(P, \sigma_n)$, equivalently $\text{Reach_committed}(P) \subseteq \text{Safe}(P)$. The encoding (proofs/induction_ghost.k) is generated by scripts/gen_ghost_artifact.py: the baseline semantics (k/srw3.k lines 24–530) is copied byte-identically — 507/507 source lines matched in order, mechanically re-verifiable (scripts/audit_ghost_faithful.py) — and the only edit class is 13 marked live lines: a <ghost> certificate cell; a $\text{GW}(I, S, \text{prev}, v)$ pending-write record in the declared-write rule; certificate transformers ghostApply/ghostUndo at the three commitment-point rules; the Ghost/GateRun sorts and ghost function signatures. The real gate decisions ($\text{allHold}(\text{applicable}(R, \text{FD}), P)$) are kept verbatim as the accept/reject splitting conditions — the certificate is purely additive.

The prescribed structure, mechanized end-to-end. GhostMatches(ghost, committed) is the explicit soundness relation — NOT an axiom: the base instance is checked by evaluation (GH-BASE); preservation is proved per transition outcome (GH-A accept / GH-R reject, the 4a/4b analogues at the certificate layer, with the chkM observer firing only if the certificate matches the committed state AND is safe); the certificate-to-real-state bridge is proved (GH-SOUND-I/F: certified-safe

matched certificate implies the real-state observer safeMin); and the certificate-predicted accept condition equals the real gate condition (GATE-AGREE).

claim	content	verdict
GH-BASE	GhostMatches base instance (by evaluation)	PROVED
GH-A	accept preservation: GhostMatches(ghost', committed') after an accepted transition	PROVED
GH-R	reject preservation: the previous matching relation is preserved	PROVED
GH-SOUND-I / F	certified-safe matched certificate $\Rightarrow$ real-state safeMin (residual-sharing / named forms)	PROVED
GATE-AGREE	certificate-predicted accept condition == real gate condition	PROVED
GH-ISO / GH-T1	non-circular diagnostics: one cycle (both branches) / two cycles	PROVED
GH-T2	THE $\forall n$ THEOREM as [circularity] over chkG-terminated GateRun programs	PROVED (#Top, no vacuity)
GH-M	strongest form: GhostMatches re-checked at every termination (chkM observer)	PROVED (#Top, no vacuity)
GH-T	minimal-destination probe (same theorem, destination mentions only $\langle k \rangle.K$)	STUCK – documents the kore behavior below
GH-T2-NEG	non-vacuity control: InitialSafe premise dropped	FAILS (correctly)

Why this is a genuine mechanization. The symbolic program tail (PG:GateRun, unbounded) means the proof can only close through the circular reuse of the claim — the induction is genuinely exercised, not unrolled. The mathematical induction was already valid (base: evaluated InitialSafe; step: $4a \wedge 4b$, machine-checked); what this pass adds is the machine-checked closure of the induction itself, with every semantic link — GhostMatches base, preservation per outcome, certificate-to-real-state soundness, gate agreement, encoding faithfulness — an independently checked proof obligation. Scope statement (precise): the $\forall n$ theorem is mechanized over the ghost-instrumented semantics, whose copied rule bodies are byte-identical to the baseline by mechanical certificate and whose certificate layer is verified conservative. The residual gap to "the literal SRW3-GATE module proves the same claim" is a module-identity artifact of the kore destination behavior below, not a mathematical step.

Prover-boundary discovery (recorded for future K users; not a theorem gap). A kore implication check fails with a stuck residual when a cell that CHANGES during the proof is absent from the claim's destination; the SAME claim with explicit $\exists$ -prefixed existential destination cells over all cells closes. GH-T (destination mentioning only $\langle k \rangle.K$) is retained as the documented probe — the same residual class as the [C] record attempt — and explains why the copied SRW3-CLAIMS claims cannot be re-run against the instrumented definition (their destinations do not mention $\langle \text{ghost} \rangle$); on the pristine definition they prove unchanged (matrix [3]). The superseded attempts [B] (fully symbolic Map: requires symbolic Map evaluation) and [C] (record mirror: destination value-linkage) remain in the repository as recorded evidence of why this encoding closes where those did not.

R.2 Secondary objective — tier-3 universal reconciliation: STATEMENT FORMALIZED; NOT YET MECHANIZED

The mandated statement, universal over registry H, declared footprint FD and prospective state S, with the exact audit-established coverage predicate $\text{wfCoverage}(H, FD, S) := \forall \text{edge}(J, Xs) \in H. (Xs \cap FD \neq \emptyset \vee \text{evalInv}(J, S))$: $\text{wfCoverage}(H, FD, S) \rightarrow \text{gateOk_faithful}(H, S) = \text{implementedGate}(H, FD, S)$. proofs/tier3_universal.k formalizes it without weakening (no coverage conjunct dropped, no quantifier restricted) and attempts mechanization; three attempts, all stuck at the same backend boundary, residuals recorded verbatim (tier3_universal.txt): T3-U ($\forall H \forall FD \forall S$) — the identity $\text{memberName}(\text{appName}(\text{Ac}), \text{Rt}) \wedge \text{allHold}(\text{universeObls}(H), S) == \text{Bool member}(\text{Ac}, \text{Rt}) \wedge \text{allHold}(\text{applicable}(H, FD), S)$ needs structural induction over constructor-free symbolic Sets; T3-FD (concrete H, $\forall FD \forall S$) — symbolic FD alone already leaves $\text{allHold}(\text{applicable}(\text{concrete-H}, FD), S)$ non-reducible; T3-A (authorization lemma alone) — pins the exact boundary: $\text{memberName}(\text{appName}(\text{Ac}), \text{As}) == \text{Bool member}(\text{Ac}, \text{As})$ requires structural induction over a symbolic Set. Classification, per mandate: STATEMENT FORMALIZED; NOT YET MECHANIZED; tier-1/2 results preserved unchanged. The bridge terminology remains "signature-faithful realization with mechanically checked ground agreement over the covered states" — never "refinement theorem".

R.3 Strengthened scientific boundary — four enforcement levels

The final report distinguishes four levels by ENFORCEMENT PHASE (complementing the by-site scheme of §Q.4, which remains the deployment-site view — site × phase):

level	what it is / can prevent	trust · bypass · consensus · independent contracts
A. Detection	observes/identifies an unsafe transition (Forta-class); prevents nothing by itself, enables reaction	whoever acts on the detection · bypassable (lag, evasion) · not consensus-critical · only as good as observability
B. Operational pre-inclusion prevention	builder/sequencer/runtime rejects the tx before settlement (Phylax-class); prevents unsafe txs from txs it chooses to process	tx submitter / order-flow source · bypassable (submit elsewhere) · no (liveness/economics) · YES relative to the enforcing inclusion set
C. Commitment-point semantic enforcement	the execution semantics contains an explicit prospective-state → obligation-check → commit/reject boundary (SRW3's modeled mechanism); prevents any state change admitted through the semantics without passing the gate	whoever runs/validates the semantics · NOT bypassable within the semantics (the gate is total over transitions) · depends where the semantics runs · YES for contracts registered in the hypergraph with complete footprints
D. Consensus/protocol validity	the validity rule itself makes obligation-violating transitions protocol-invalid; prevents all violations by all validators	all validators / social consensus · NOT bypassable (violation = invalid block) · YES · YES network-wide for covered contracts

The levels do NOT have identical trust assumptions, and the strongest SRW3 theorem states exactly which level it assumes. The mechanized theorem (GH-T2/GH-M) is a theorem about LEVEL C: it quantifies over transitions admitted by the semantics' commitment gate, with premises InitialSafe (evaluated), SecurityClosed (evaluated closure) and the declared-footprint one-write cycle program class; it makes no claim about transactions that never enter the gate. Relative to level B, the same gate gives safety for every tx in the enforcing inclusion set that settles. Consensus-strength (level D) is NOT a result of this phase: it requires the commitment-point hook embedded in a consensus-validity rule — REQUIRES CLIENT/PROTOCOL SUPPORT (§H.3/§Q.4).

R.4 Prior-art position — frozen (unchanged after the freeze check)

The corrected Round-2 position (§Q.3) is retained verbatim and now FROZEN: pre-inclusion invariant enforcement is NOT novel (Phylax Credible Layer and peers); cross-contract invariant checking is NOT novel; formal invariant verification is NOT novel; lineage/hash chains are NOT novel. The candidate contribution remains the CONJUNCTION: explicit interaction hypergraph + closure-completeness condition + effect-completeness requirement + commitment-gated composition + lineage structure + executable mechanization over KEVM + compositional safety theorem. Per the freeze mandate, three targeted searches were run against the conjunction before freezing (raw evidence: tool-results/prior-art/freeze/, 2026-09-28): guard-contract post-state assertion patterns (contract-level checks; no hypergraph/closure/lineage/mechanization), constraint-based speculative execution (database equivalence checking; no invariant-gated commitment), and declarative safety verifiers (VerX/DCV-class; static, not commitment-gated). No surveyed system combines substantially all seven elements. Had a close system been found, the claim would have been narrowed again rather than defended; none was.

R.5 Phase-0-D completion classification (mandated 17 items)

#	item	classification
1	Claims 1–3 (local preservation; gate coverage; lineage append)	PROVED BY K
2	Claim 4a (admissible step preserves Safe)	PROVED BY K
3	Claim 4b (inadmissible step cannot commit)	PROVED BY K
4	full $\forall n$ safety theorem	PROVED BY K — mechanized over the ghost-instrumented semantics (GH-T2/GH-M #Top; GhostMatches base/preservation/soundness + gate agreement machine-checked; faithfulness certificate 507/507; non-vacuity control fails correctly)
5	tier-1 reconciliation (ground agreement)	PROVED BY K (checked-by-evaluation class)

#	item	classification
6	tier-2 generalized reconciliation (state-generalized under wfCoverage)	PROVED BY K
7	tier-3 universal reconciliation ($\forall H \forall FD \forall S$)	STATEMENT FORMALIZED; NOT YET MECHANIZED (symbolic-set induction boundary; statement not weakened)
8	effect completeness (wfCheck: $\Gamma^* \subseteq \Gamma^\wedge$)	PROVED BY K (enforced by the semantics; trust-policy divergence pinned to CM7/X1)
9	interaction closure (InteractionComplete/ContractComplete)	PROVED BY K (evaluated closure; CM1/CM6 countermodels demonstrate necessity)
10	structural lineage ($\wedge$ records; single commit path; head advance)	PROVED BY K (Claims 2/3 + rule census)
11	cryptographic lineage (Verify(parent, data, proof))	NOT YET MECHANIZED (keccak, state-root binding, MPT proofs, signatures)
12	KEVM execution binding (real SSTORE/CALL/RETURN; obligations over real storage; commit/restore)	DEMONSTRATED BY KEVM
13	commitment-point mechanism (prospective $\rightarrow$ obligation-check $\rightarrow$ commit/reject)	PROVED BY K (only the gate accept rule updates committed; Claim 2; the $\forall n$ theorem rides it)
14	CM2 (aliasing across storage namespaces)	NOT YET MECHANIZED (modeled single namespace)
15	CM3 (aggregate-authority limits)	NOT YET MECHANIZED
16	CM8 (hyperproperty-level guarantees)	NOT YET MECHANIZED
17	CM9 (liveness)	NOT YET MECHANIZED

plus: commitment-point hook in an execution client / protocol — REQUIRES CLIENT/PROTOCOL SUPPORT (§H.3/§Q.4/§R.3); obligation registries and A1–A3 abstractions — ASSUMED per model (explicit, §E).

R.6 Regression matrix v3 — nothing changed

#	Check	Result (full_matrix_v3.txt)
1	Baseline Python suite (make test, canonical)	Ran 15 tests ... OK – exit 0
2	Abstract demo suite	13/13 as-expected – exit 0
3	Claims 1, 2, 3, 4a, 4b (kprove)	exit 0, #Top, no vacuity
4	Negative control	exit 113 – correctly fails
5	Reconciliation bridge (9 ground claims)	exit 0, #Top, 9/9 checked-by-evaluation (fresh kompiled)
6	Phantom-reject defect audit	corrected = deterministic commit; defective = spurious 2nd outcome
7a/7b/7c	Round-2 artifacts	safe-form 4/4 #Top; generalized bridge 5/5 #Top; necessity probe correctly FAILS
8	KEVM positive / stale / minimal / min2	positive = COMMIT {oracle:{0:100,1:1}, lending:{0:100,1:5}, liq:{0:4,1:100}} + lineage price 100; stale = REJECT + full restore + no lineage (n=0, h=-1); minimal = setup-only smoke demo; min2 = REJECT + restore (n=0, h=-1) – demo retargeted from pre-renaming account 1 to 4097 (w30racle) with changelog note; verdict semantics identical to the round-1 canonical record
9	Ghost encoding (fresh kompiled + re-run)	faithfulness certificate 507/507 PASS; all 12 per-claim verdicts identical to the discovery run (GH-T stuck as documented; GH-T2-NEG fails as designed)
10	Tier-3 attempts (fresh kompiled + re-run)	same three stuck residuals – STATEMENT FORMALIZED; NOT YET MECHANIZED
11	Prior-art freeze evidence	present (tool-results/prior-art/freeze/)

No previously established result changed.

R.7 LLVM decision — resolved

Per the mandate’s rule: the full $\forall n$ theorem IS mechanized (GH-T2/GH-M reach #Top), so this pass’s outcome is the first branch: the theorem-closure result is recorded as a major Phase-0-D result, and LLVM backend work is now UNBLOCKED as the next phase’s first engineering step. The backend boundaries documented along the way (symbolic-Map evaluation in [B]; destination-value linkage in [C] and GH-T; symbolic-set induction in T3-U/T3-FD/T3-A) are recorded verbatim so LLVM retargeting can be evaluated against exactly these packaging

limitations. The tier-3 statement remains formalized and unweakened; if a future backend upgrade mechanizes it, the tier ladder completes without any change to the statements.